

第三章

socket编程

socket简单了解

◆什么是socket？

- (1) 对TCP/IP的一层封装，程序员可以通过该API实现网络通信。
- (2) 用于描述IP地址和端口，是一个逻辑通信链的句柄。
- (3) 在Internet上的主机一般运行了多个服务软件，同时提供几种服务。每种服务都打开一个Socket，并绑定到一个端口上，不同的端口对应于不同的服务。

为什么要使用socket

- ◆ 普通的I/O操作过程
 - ◆ 打开文件—>读/写操作—>关闭文件
- ◆ TCP/IP协议被集成到操作系统的内核中，引入了新型的“I/O”操作
 - ◆ 进行网络通信的两个进程在不同的机器上，如何连接？
 - ◆ 网络协议具有多样性，如何进行统一的操作
- ◆ 需要一种**通用**的网络编程接口：Socket

socket套接字的类型

◆流式套接字(SOCK_STREAM)

- ◆ 提供了一个面向连接、可靠的数据传输服务，数据无差错、无重复的发送且按发送顺序接收。内设置流量控制，避免数据流淹没慢的接收方。数据被看作是字节流，无长度限制。

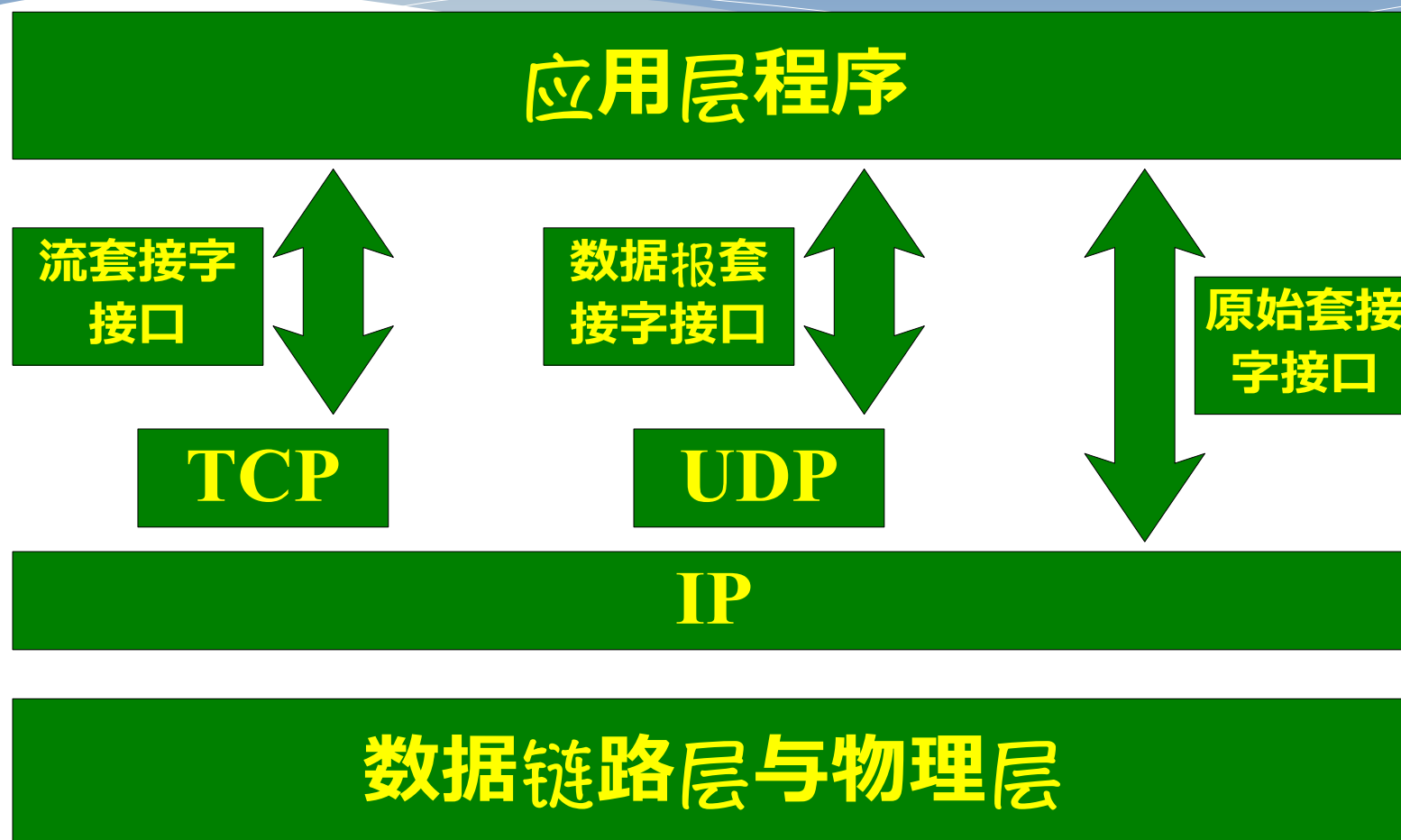
◆数据报套接字(SOCK_DGRAM)

- ◆ 提供无连接服务。数据包以独立数据包的形式被发送，不提供无差错保证，数据可能丢失或重复，顺序发送，可能乱序接收。

◆原始套接字(SOCK_RAW)

- ◆ 可以对较低层次协议如IP、ICMP直接访问。

socket的位置



Ip地址

- ◆ IP地址是Internet中主机的标识
 - ◆ Internet中的主机要与别的机器通信必须具有一个IP地址
 - ◆ IP地址为32位（IPv4）或者128位（IPv6）
 - ◆ 每个数据包都必须携带目的IP地址和源IP地址，路由器依靠此信息为数据包选择路由
- ◆ 表示形式：常用点分形式，如202.38.64.10，最后都会转换为一个32位的无符号整数。

Ip 的地址的转换

◆inet_aton()

- ◆将strptr所指的字符串转换成32位的网络字节序二进制值

#include <arpa/inet.h>

```
int inet_aton(const char *strptr, struct in_addr *addrptr);
```

◆inet_addr()

- ◆功能同上，返回转换后的地址。

```
int_addr_t inet_addr(const char *strptr);
```

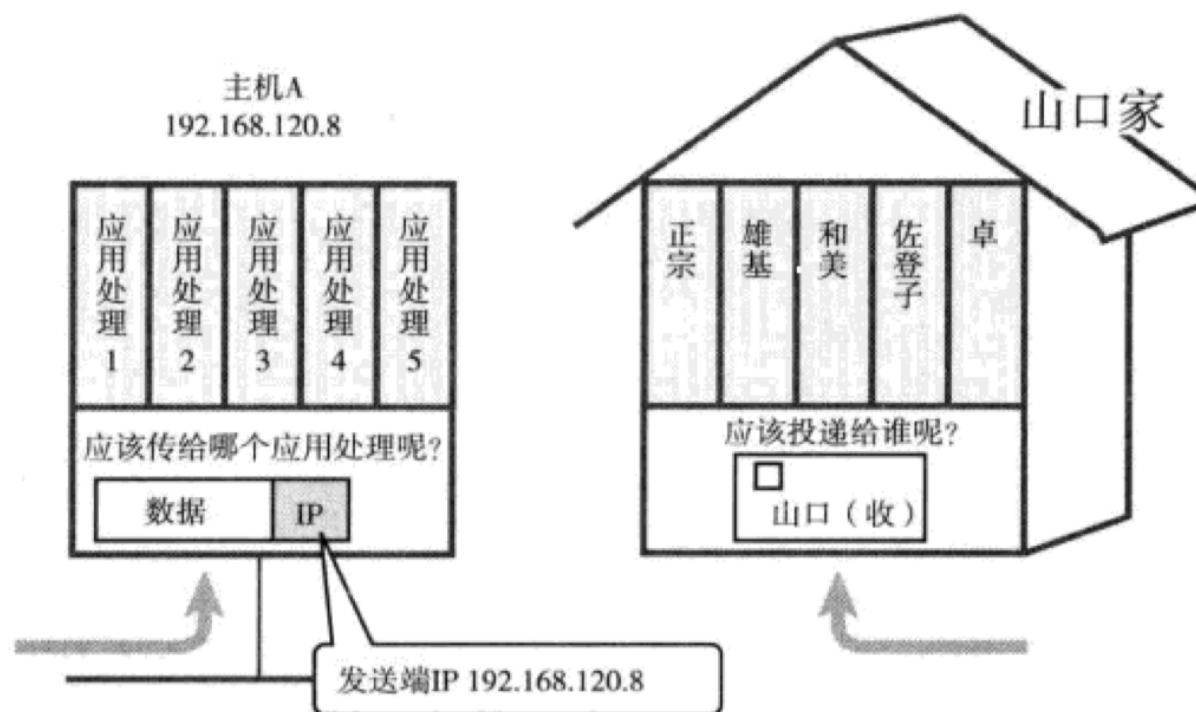
◆inet_ntoa()

- ◆将32位网络字节序二进制地址转换成点分十进制的字符串。

```
char *inet_ntoa(struct in_addr inaddr);
```

端口号

思考：一个电脑装有很多应用程序，那么怎么知道是哪个应用程序去接收数据？

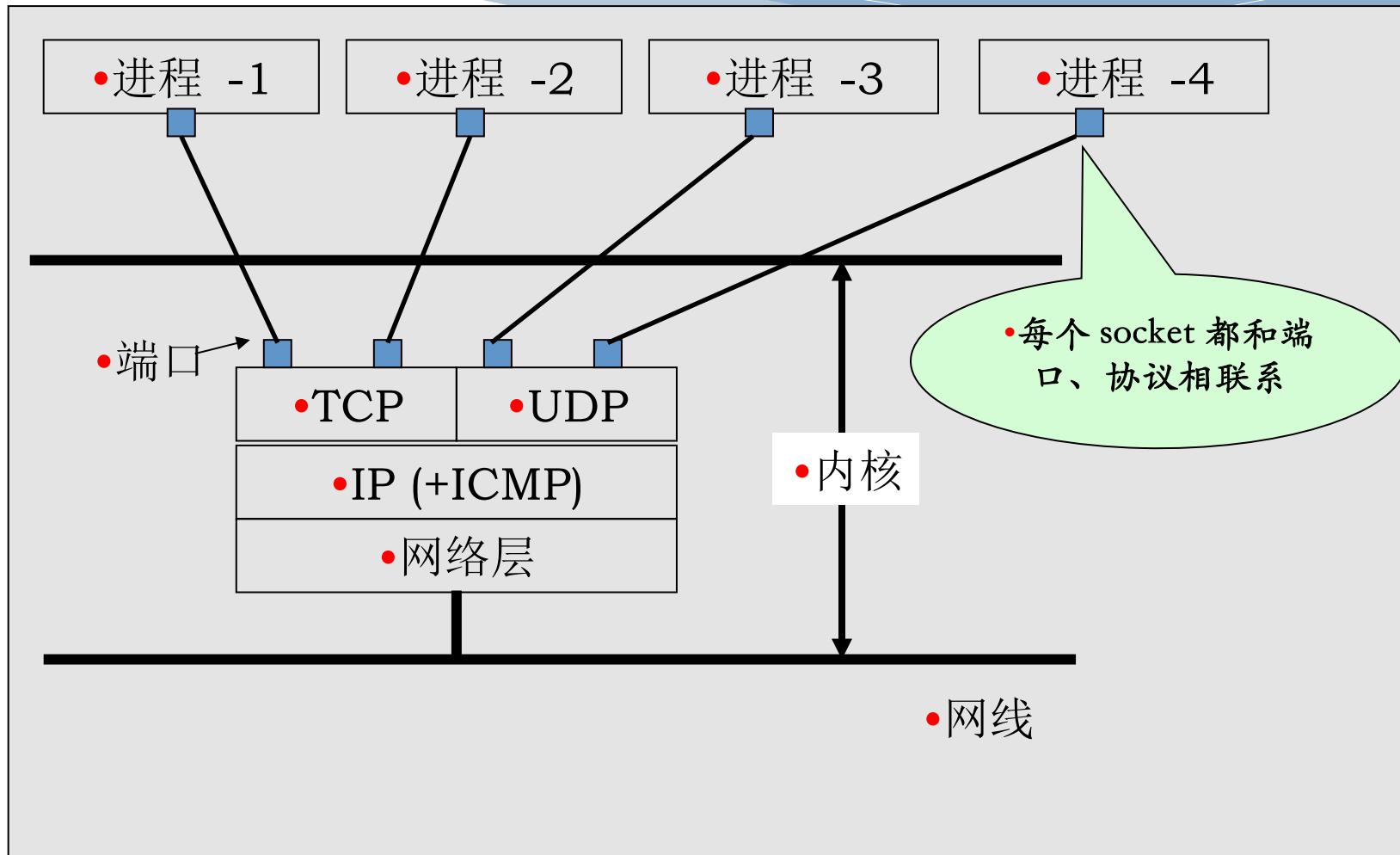


通常一台计算机中运行着多个应用程序，
因此有必要将其区分进行识别。

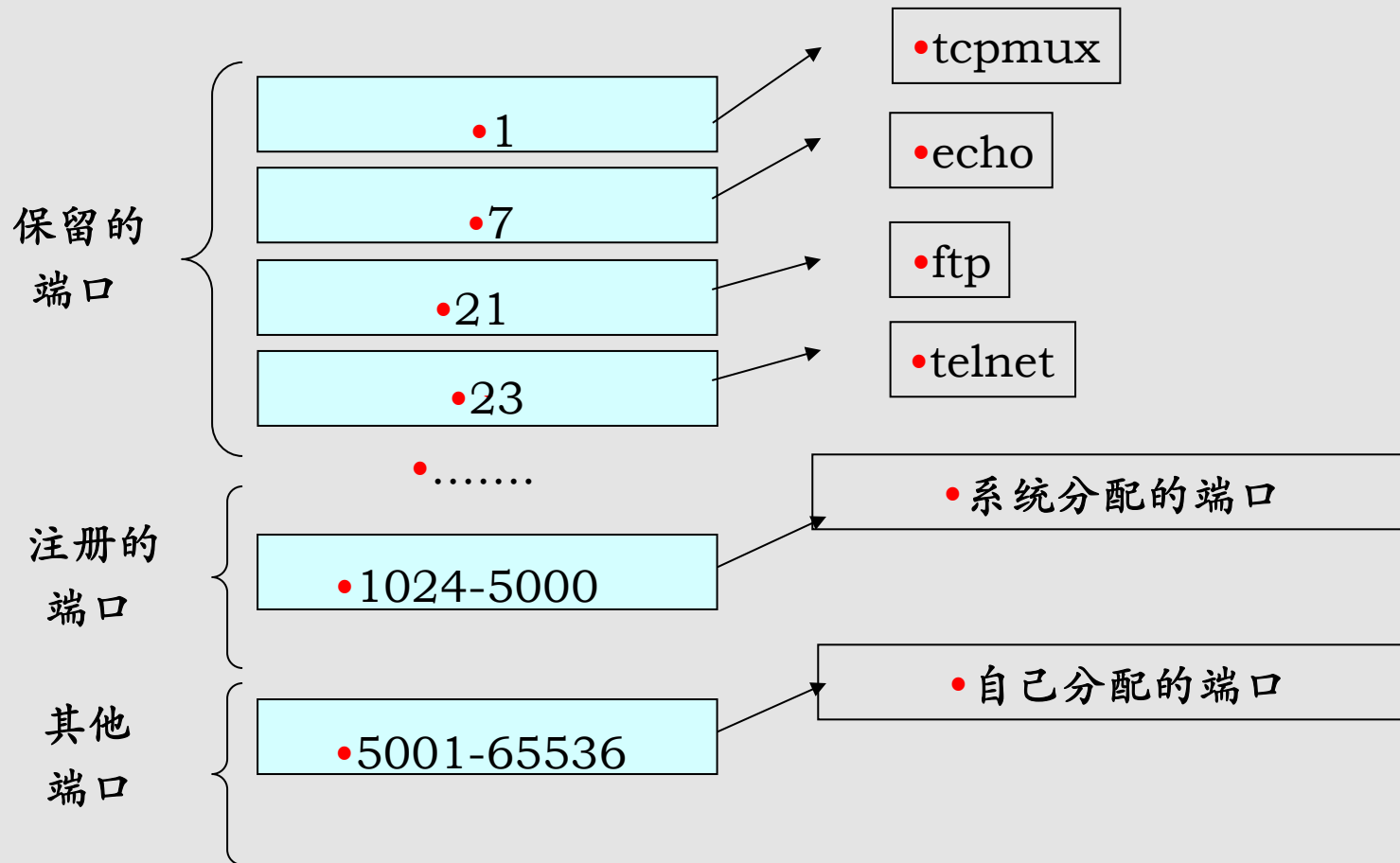
端口号

- ◆ 为了区分一台主机接收到的数据包应该转交给哪个进程来处理，使用端口号来区别
- ◆ TCP端口号与UDP端口号独立
- ◆ 端口号一般由IANA (Internet Assigned Numbers Authority) 管理
 - ◆ 众所周知端口：1~1023（1~255之间为众所周知端口，256~1023端口通常由UNIX系统占用）
 - ◆ 注册端口：1024~49150
 - ◆ 动态或私有端口：49151~65535

套接字和端口



套接字和端口



一个比喻

- ◆如果把网络数据包的投递过程看成是给远方的一位朋友寄一封信，那么：
 - ◆IP地址就是这位朋友的所在位置，如上海交大XX系，邮局依靠此信息进行信件的投递，网络数据则依靠IP地址信息进行路由
 - ◆端口号就是这位朋友的名字，传达室依靠这个信息最终把这封信交付给这位收信者，数据包则依靠端口号送达给接收进程

字节序

- ◆ 不同类型CPU的主机中，内存存储多字节整数序列有两种方法，称为主机字节序(HBO):
 - ◆ 小端序（little-endian） - 低序字节存储在低地址
 - ◆ 将低字节存储在起始地址，称为“Little-Endian”字节序，Intel、AMD等采用的是这种方式；
 - ◆ 大端序（big-endian） - 高序字节存储在低地址
 - ◆ 将高字节存储在起始地址，称为“Big-Endian”字节序，由ARM、Motorola等所采用
- ◆ 网络中传输的数据必须按网络字节序，即大端字节序

字节序

- ◆网络字节序(NBO - Network Byte Order)

- ◆使用统一的字节顺序，避免兼容性问题

- ◆主机字节序(HBO - Host Byte Order)

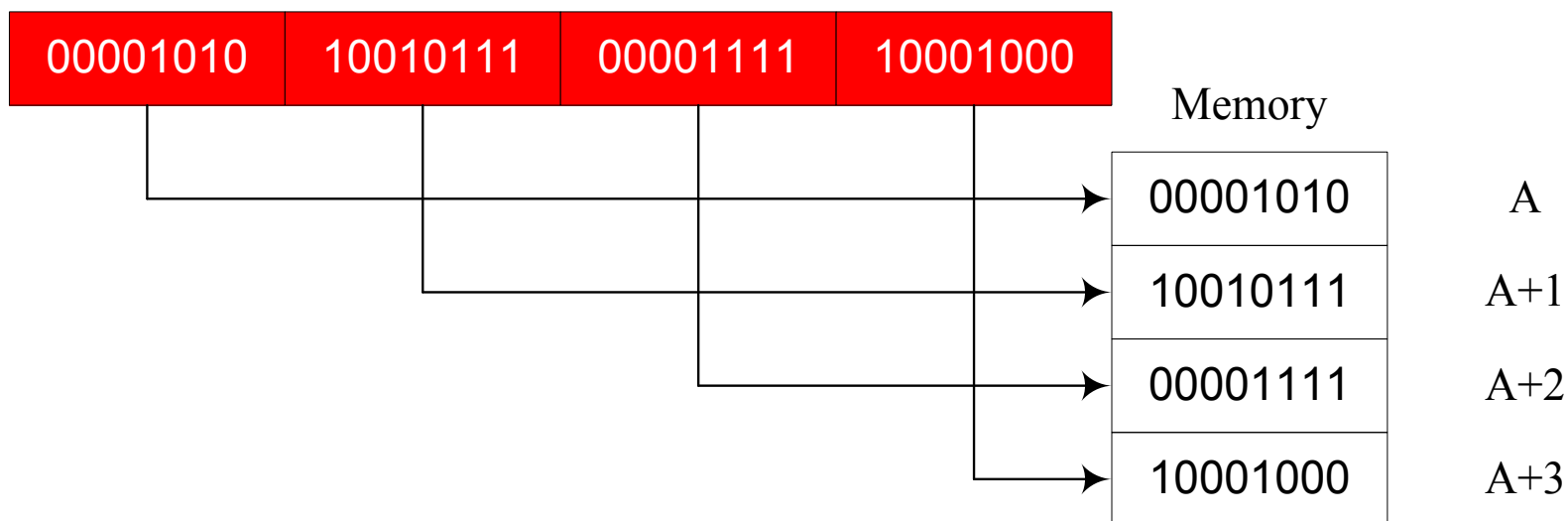
- ◆不同的机器HBO是不一样的，这与CPU的设计有关

- ◆Motorola 68K系列、ARM系列，HBO与NBO是一致的

- ◆Intel X86系列，HBO与NBO不一致

字节序

- ◆ 大端(Big-Endian):字节的高位在内存中放在存储单元的起始位置



- ◆ 小端(Little-Endian):与大端相反

示例代码: 检测大小端

```
#include<stdio.h>
Int main()
{
    int a=0x01234567;
    char *p=(char *)&a;
    printf(“%d\n”,*p);
}
//观察输出
```

字节序转换函数

- ◆把给定系统所采用的字节序称为主机字节序。为了避免不同类别主机之间在数据交换时由于对于字节序的不同而导致的差错，引入了网络字节序。

- ◆主机字节序到网络字节序

- ◆`u_long htonl (u_long hostlong);`

- ◆`u_short htons (u_short short);`

- ◆网络字节序到主机字节序

- ◆`u_long ntohl (u_long hostlong);`

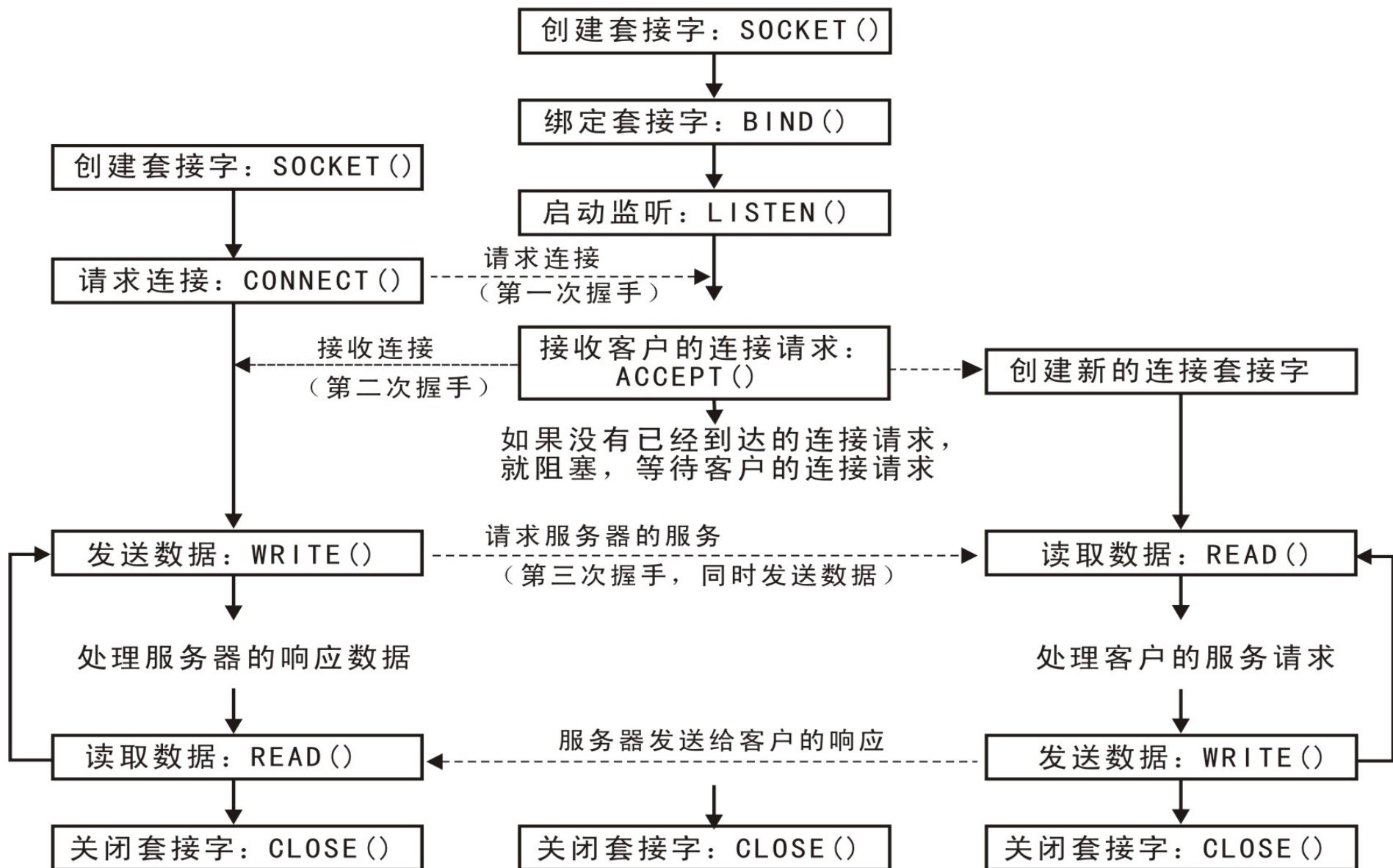
- ◆`u_short ntohs (u_short short);`

客户端

服务器端

对于监听套接字

对于连接套接字



网络编程的API

◆ 网络编程常用函数

- ◆ `socket()` 创建套接字
- ◆ `bind()` 绑定本机地址和端口
- ◆ `connect()` 建立连接
- ◆ `listen()` 设置监听端口
- ◆ `accept()` 接受TCP连接
- ◆ `recv()`, `read()`, `recvfrom()` 数据接收
- ◆ `send()`, `write()`, `sendto()` 数据发送
- ◆ `close()`, `shutdown()` 关闭套接字

socket创建套接字

◆ `int socket (int domain, int type, int protocol);`

◆ domain 是地址族

`AF_INET` // internet 协议

`AF_UNIX` // unix internal协议

◆ type // 套接字类型

`SOCK_STREAM` // 流式套接字

`SOCK_DGRAM` // 数据报套接字

`SOCK_RAW` // 原始套接字

◆ protocol 参数通常置为0

socket () 函数

returns socket ID on success
return -1 on error

```
[int socket_fd] = socket (AF_INET, SOCK_STREAM, 0);
```

“AF_INET” = 使用网络协议族

“SOCK_STREAM” = 使用 TCP

总是 0

地址相关的数据结构

◆ 通用地址结构

```
struct sockaddr
{
    u_short sa_family; // 地址族, AF_xxx
    char sa_data[14]; // 14字节协议地址
};
```

◆ Internet 协议地址结构

```
struct sockaddr_in
{
    u_short sin_family; // 地址族, AF_INET, 2 bytes
    u_short sin_port; // 端口, 2 bytes
    struct in_addr sin_addr; // IPV4地址, 4 bytes
    char sin_zero[8]; // 8 bytes unused, 作为填充
};
```

地址相关的数据结构

◆ IPv4地址结构

// internet address

struct in_addr

{

in_addr_t s_addr; // u32 network address

};

bind绑定端口

◆ `int bind(int sockfd, struct sockaddr *my_addr, int addrlen);`

◆ 头文件:

`#include <sys/types.h>`

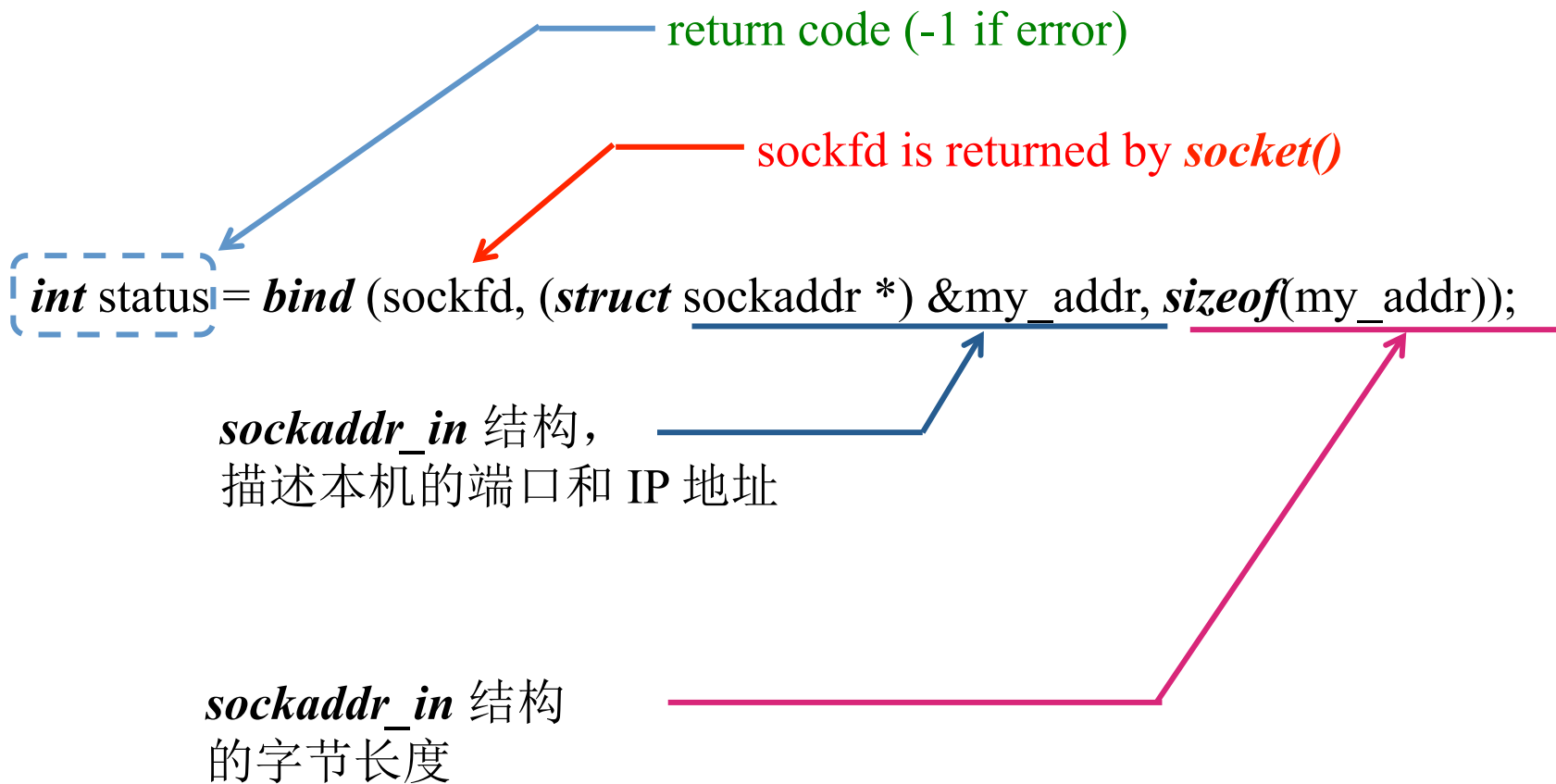
`#include <sys/socket.h>`

◆ `sockfd`: `socket`调用返回的文件描述符

◆ `addrlen`: `sockaddr`地址结构的长度

◆ 返回值: 0或-1

bind () 函数



bind绑定端口

```
int bind (int sockfd, struct sockaddr* addr, int addrLen);
```

◆ **sockfd** 由socket() 调用返回

◆ **addr** 是指向 sockaddr_in 结构的指针，包含本机IP 地址和端口号

struct sockaddr_in

u_short sin_family // protocol family

u_short sin_port // port number

struct in_addr sin_addr //IP address (32-bits)

◆ **addrLen** : sizeof (struct sockaddr_in)

地址信息初始化

Step 1: 初始化该数据结构

```
struct sockaddr_in my_addr;
```

•
•
•

```
my_addr.sin_family = AF_INET;      /* 协议*/  
my_addr.sin_port = htons (6666);  /* 端口号*/  
my_addr.sin_addr.s_addr = inet_addr("192.168.1.100"); /* IP地址*/
```

Step 2: 填充信息

地址转换函数

◆ unsigned long inet_addr(char *address);

address是以NULL结尾的点分IPv4字符串。该函数返回32位的地址。如果字符串包含的不是合法的IP地址，则函数返回-1。例如：

```
struct in_addr addr;  
addr.s_addr = inet_addr(" 192.168.1.100 ");
```

◆ char* inet_ntoa(struct in_addr address);

address是IPv4地址结构，函数返回一指向包含点分IP地址的静态存储区字符指针。如果错误则函数返回NULL

◆ int inet_aton(const char *string, struct in_addr *addr);

将一个字符串IP地址转换为一个32位的网络序列IP地址

listen 监听外部连接

*** int listen (int sockfd, int backlog);**

*** sockfd:** 监听连接的套接字

*** backlog**

- 指定了正在等待连接的最大队列长度，它的作用在于处理可能同时出现的几个连接请求。
- 返回值： 0 或 -1

完成listen()调用后，socket变成了监听socket(listening socket).

accept接收请求连接

◆ `int accept(int sockfd, struct sockaddr *addr, socklen_t *addrlen);`

◆ 返回值：已建立好连接的套接字或-1

◆ 头文件

`#include <sys/types.h>`

`#include <sys/socket.h>`

◆ `sockfd`：监听套接字

◆ `addr`：对方地址

◆ `addrlen`：地址长度

`listen()`和`accept()`是TCP服务器端使用的函数

accept () 函数

int new_accepted_fd = *accept* (listen_fd, (struct *sockaddr* *) client_addr, client_addr_length);

一个新的已连接的socket
(-1 if error)

接受客户连接的socket, 即
listening socket

接收外来连接的地址信息, 如果不关心, 可置为NULL

指针: 接收连接的sockaddr_in结构体的长度

connect 客户端发送连接请求

◆ `int connect(int sockfd, struct sockaddr *serv_addr, int addrlen);`

◆ 返回值：0 或 -1

◆ 头文件：

`#include <sys/types.h>`

`#include <sys/socket.h>`

◆ `sockfd` : socket返回的文件描述符

◆ `serv_addr` : 服务器端的地址信息

◆ `addrlen` : `serv_addr`的长度

connect()是客户端使用的系统调用。

connect() 函数

int status = *connect* (socket_fd, (*struct* sockaddr *) &addr, *sizeof*(addr));

return code (-1 if error)

socket_fd returned by *socket()* system call

struct sockaddr 结构,
描述服务器的端口和 IP
地址

struct sockaddr
结构的字节长度

The diagram illustrates the `connect()` function signature: `int status = connect(socket_fd, (struct sockaddr *) &addr, sizeof(addr));`. Annotations include: a blue arrow pointing to the return value `int status` with the text "return code (-1 if error)"; a red arrow pointing to the `socket_fd` parameter with the text "socket_fd returned by *socket()* system call"; a blue arrow pointing to the `(struct sockaddr *) &addr` parameter with the text "*struct* sockaddr 结构, 描述服务器的端口和 IP 地址"; and a pink arrow pointing to the `sizeof(addr)` parameter with the text "*struct* sockaddr 结构的字节长度".

send 发送数据

- ◆ `ssize_t send(int socket, const void *buffer, size_t length, int flags);`
 - ◆ 返回值:
 - ◆ 成功: 实际发送的字节数
 - ◆ 失败: -1, 并设置 `errno`
 - ◆ 头文件:
 - ◆ `#include <sys/socket.h>`
 - ◆ `buffer`: 发送缓冲区首地址
 - ◆ `length`: 发送的字节数
 - ◆ `flags`: 发送方式 (通常为0)

send () 函数

on success, the number of bytes actually sent
return code (-1 if error)

socket returned by `socket()`
or `accept()`

`int status` = *send* (`socket_fd`, `out_buffer`, `MAX_BUFFER_SIZE`, 0);

待发送数据缓冲区的指针

The maximum buffer size

Always 0

recv 接收数据

- ◆ `ssize_t recv(int socket, const void *buffer, size_t length, int flags);`
- ◆ 返回值:
 - ◆ 成功: 实际接收的字节数
 - ◆ 失败: -1, 并设置 `errno`
- ◆ 头文件:
 - ◆ `#include <sys/socket.h>`
- ◆ `buffer`: 发送缓冲区首地址
- ◆ `length`: 发送的字节数
- ◆ `flags`: 接收方式 (通常为0)

recv () 函数

on success, the number of bytes received
return code (-1 if error)

Socket fd returned
by socket() call

```
[int status] = recv (socket_fd, in_buffer, MAX_BUFFER_SIZE, 0);
```

接收数据的缓冲区的指针

Example: *char* in_buffer [MAX_BUFFER]

The maximum buffer size

Always 0

read/write

◆ `ssize_t read(int fd, void *buf, size_t count);`

◆ `ssize_t write(int fd, const void *buf, size_t count);`

- `read()`和`write()`经常会代替`recv()`和`send()`，通常情况下，看程序员的偏好
- 使用`read()/write()`和`recv()/send()`时最好统一

套接字关闭

- ◆ `int close(int sockfd);`
- ◆ 关闭双向通讯

练习

练习：写一个TCP服务器，使用telnet测试其连接，再写一个客户端，测试两端的通信

端口的重复使用

◆为什么要进行端口重复使用？

当程序再次重启的时候，我们发现刚才利用的端口，已被绑定，这个会给我们测试代码的时候带来不必要的麻烦。所以我们需要对端口进行重复利用。

setsockopt设置端口重用

函数原型： `int setsockopt(int, int, int, const void *, socklen_t);`

参数： 参数一： 指定套接字描述符

参数二： 套接字类型的选项 SOL_SOCKET基本套接口

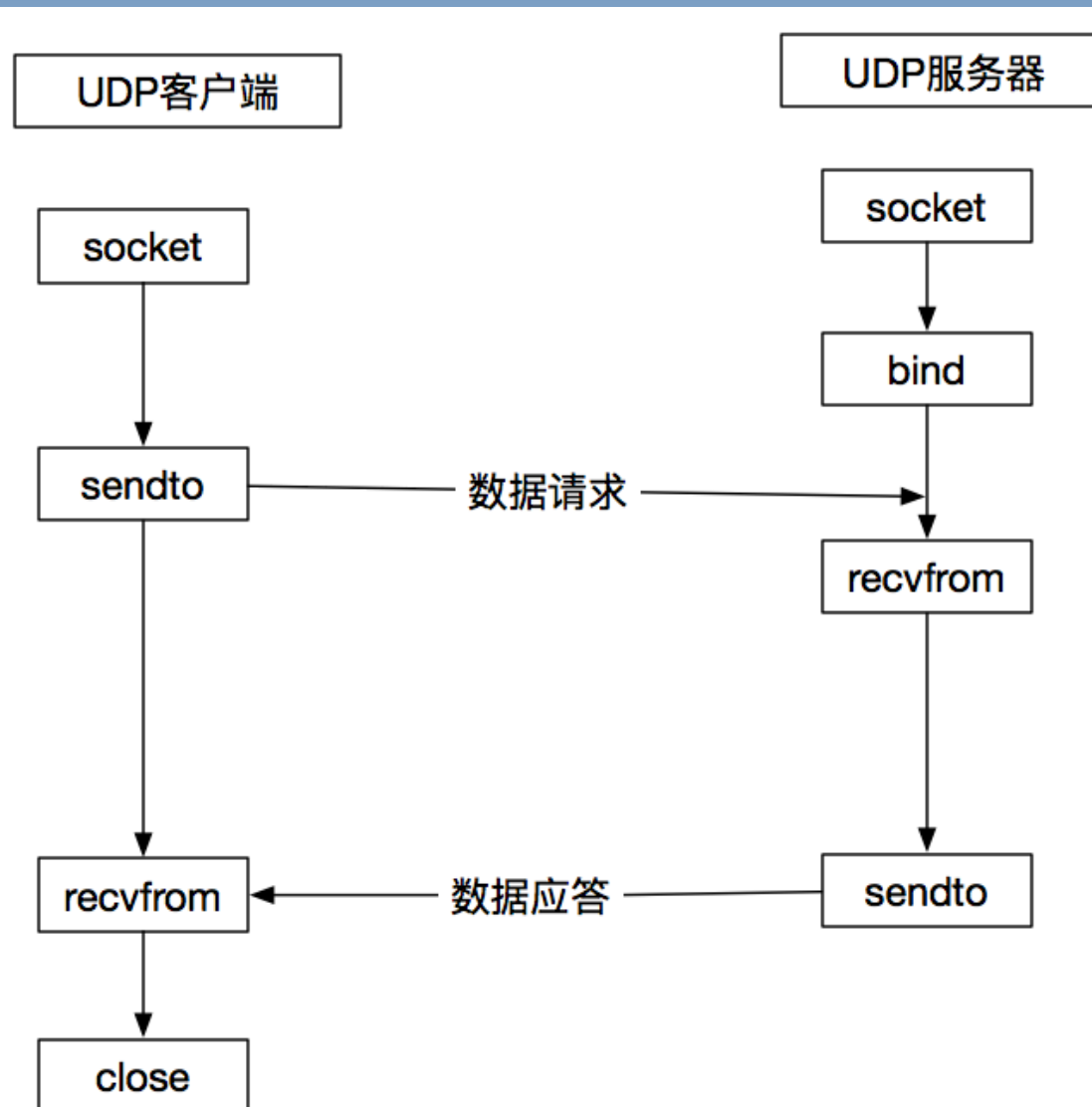
参数三： 具体功能需求，可以查看man手册

参数四： 设置打开还是关闭；不等于0时，打开调试信息，否则，关闭

参数五： 第四个参数值所占的空间大小

注意点： 在绑定端口之前设置

socket基于UDP(未连接)协议编程



sendto/recvfrom

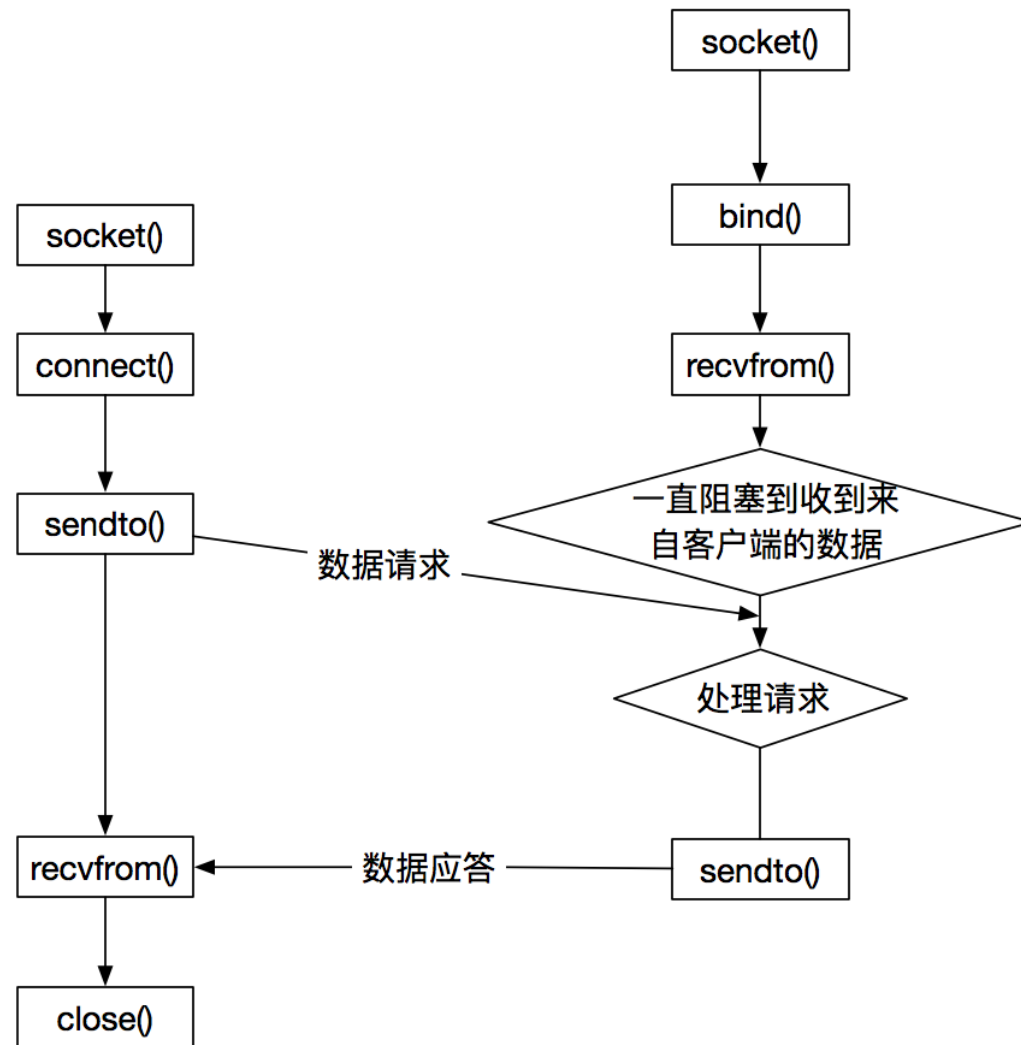
- ◆ `ssize_t sendto(int socket, void *message, size_t length, int flags, struct sockaddr *dest_addr, socklen_t dest_len);`
- ◆ `ssize_t recvfrom(int socket, void *buffer, size_t length, int flags, struct sockaddr *address, socklen_t *address_len);`

这两个函数一般在使用UDP协议时使用

练习

练习：写一个UDP（未连接）服务器，再写一个客户端，测试两端的通信

socket基于UDP(连接)协议编程



练习

练习：写一个UDP（连接）服务器，再写一个客户端，测试两端的通信

思考

◆思考：一个聊天软件，发送信息的同时，也能接收信息。发送信息监听的是标准输入，接收是监听的套接字。使用以前所学的知识解决当前的问题？

思路一：开进程

思路二：开线程

思路三：select 多路复用

select 多路复用服务器

◆ 什么是select多路复用?

- ◆ 内核一旦发现进程指定的一个或者多个IO条件准备读取, 它就通知该进程

函数原型: `int select(int n, fd_set *read_fds, fd_set *write_fds, fd_set *except_fds, struct timeval *timeout);`

头文件: `#include <sys/time.h>`

`#include <sys/types.h>`

`#include <unistd.h>`

select参数

◆ maxfd

- ◆ 所有监控的文件描述符中最大的那一个加1

◆ read_fds

- ◆ 所有要读的文件描述符的集合

◆ write_fds

- ◆ 所有要写的文件描述符的集合

◆ except_fds

- ◆ 其他要向我们通知的文件描述符

◆ timeout

- ◆ 超时设置.
- ◆ Null: 一直阻塞, 直到有文件描述符就绪或出错
- ◆ 时间值为0: 仅仅检测文件描述符集的状态, 然后立即返回
- ◆ 时间值不为0: 在指定时间内, 如果没有事件发生, 则超时返回。

select常用的宏

◆在我们调用select时进程会一直阻塞直到以下的一种情况发生.

- ◆有文件可以读.
- ◆有文件可以写.
- ◆超时所设置的时间到.

◆为了设置文件描述符我们要使用几个宏:

- ◆ `void FD_SET(int fd, fd_set *fdset)` 将fd加入到fdset
- ◆ `void FD_CLR(int fd, fd_set *fdset)` 将fd从fdset里面清除
- ◆ `void FD_ZERO(fd_set *fdset)` 从fdset中清除所有的文件描述符
- ◆ `int FD_ISSET(int fd, fd_set *fdset)` 判断fd是否在fdset集合中

select使用场景

- ◆ 当客户处理多个描述符时（一般是交互式输入和网络套接口）
- ◆ 如果一个TCP服务器既要处理监听套接口，又要处理已连接套接口，一般也要用到I/O复用。
- ◆ 如果一个服务器既要处理TCP，又要处理UDP，一般要使用I/O复用。
- ◆ 如果一个服务器要处理多个服务或多个协议，一般要使用I/O复用。

练习

练习1: 写一个客户端select多路复用处理套接字的监听和标准输入描述符的改变

练习2: 写一个服务器select多路复用处理连接监听套接字和数据通信套接字的改变